

판다스(Pandas) Series

추가 / 삭제 / 수정 & 연산 강의 교재

광명코딩학원 | 파이썬 판다스 기초 강의

[핵심 개념] Series에 값을 추가/삭제/수정할 수 있고, 브로드캐스팅, 사칙연산, 비교연산, 필터링 등 강력한 연산 기능을 제공합니다.

PART 1. Series 값 추가 / 삭제 / 수정

1. Series에 값 추가

[개념] 딕셔너리와 유사한 방식으로 새로운 인덱스에 값을 할당하면 추가됩니다. 해당 인덱스가 없으면 추가, 있으면 수정으로 동작합니다.

값 추가 예제

```
import pandas as pd

data = [100, 200, 300]

index = ['월', '화', '수']

s = pd.Series(data, index)

# 방법 1: loc 사용 (권장)

s.loc['목'] = 400

# 방법 2: 대괄호 직접 사용

s['금'] = 500

print(s)
```

```
# 월 100

# 화 200

# 수 300

# 목 400

# 금 500

# dtype: int64
```

방법	문법	설명
loc 사용 (권장)	s.loc['목'] = 400	명시적 인덱스 지정 — 권장 방식
대괄호	s['금'] = 500	간편하게 바로 추가 가능

[핵심] 추가 vs 수정 구분: 해당 인덱스가 없으면 → 추가 / 이미 있으면 → 수정

2. Series 값 삭제 (drop)

[개념] drop() 메서드로 값을 삭제합니다. 기본적으로 원본은 유지되고 삭제된 새로운 시리즈를 리턴합니다. inplace=True를 사용해야 원본이 변경됩니다.

drop 예제

```
data = [100, 200, 300]

index = ['월', '화', '수']

s = pd.Series(data, index)

# 기본: 원본 유지, 새 시리즈 리턴

s1 = s.drop('월')

print(s1) # 화 200 / 수 300

print(s) # 원본 그대로: 월 100 / 화 200 / 수 300

# inplace=True: 원본 직접 변경
```

```
s.drop('목', inplace=True)

print(s) # 목이 삭제된 원본

# 여러 개 동시 삭제 (리스트 사용)

s2 = s.drop(['수', '금'])

print(s2) # 월 100 / 화 200
```

방법	문법	원본 변경
기본 drop	s1 = s.drop('월')	X — 원본 유지, 새 시리즈 리턴
inplace=True	s.drop('목', inplace=True)	O — 원본 직접 변경
여러 개 삭제	s.drop(['수', '금'])	X — 원본 유지

[주의] [주의] drop()은 기본적으로 원본을 변경하지 않습니다! 원본을 바꾸려면 반드시 inplace=True 를 추가하세요.

3. Series 값 수정

[개념] 기존 인덱스에 새 값을 할당하면 수정됩니다. iloc(행 번호), loc(인덱스 레이블), 대괄호[] 세 가지 방법을 모두 사용할 수 있습니다.

값 수정 예제

```
data = [100, 200, 300]

index = ['월', '화', '수']

s = pd.Series(data, index)

# 방법 1: 대괄호 사용

s['수'] = 3000

print(s) # 월:100 화:200 수:3000

# 방법 2: iloc (행 번호로 수정)

s.iloc[2] = 4000
```

```
print(s) # 월:100 화:200 수:4000
```

```
# 방법 3: loc (인덱스 레이블로 수정 — 권장)
```

```
s.loc['수'] = 5000
```

```
print(s) # 월:100 화:200 수:5000
```

방법	문법	기준
대괄호	s['수'] = 3000	인덱스 레이블
iloc	s.iloc[2] = 4000	행 번호
loc (권장)	s.loc['수'] = 5000	인덱스 레이블 — 명시적, 권장

PART 2. Series 연산

4. 브로드캐스팅 (Broadcasting)

[개념] 반복문 없이 Series의 모든 원소에 동시에 연산이 적용되는 기능입니다. 마치 엑셀에서 수식 하나를 드래그하면 전체 셀에 적용되는 것과 같습니다.

브로드캐스팅 예제

```
from pandas import Series

s = Series([100, 200, 300])

print(s + 10) # [110, 210, 310]

print(s * 2) # [200, 400, 600]

print(s - 50) # [ 50, 150, 250]

print(s / 10) # [ 10.0, 20.0, 30.0]

# 결과 예시 (s + 10)

# 0 110

# 1 210

# 2 310

# dtype: int64
```

- 연산이 Series 객체의 전체 값에 동시에 적용됨
- 반복문(for문)을 사용하지 않아도 됨
- 파이썬 리스트에는 없는 기능 — 넘파이 기반으로 동작

5. 사칙연산 — 두 Series 간 연산

5 - 1. 같은 인덱스끼리 연산 (정수 인덱스)

사칙연산 — 정수 인덱스

```

from pandas import Series

high = Series([51500, 51200, 52500, 51500, 51500])

low = Series([50700, 50500, 50500, 50800, 50700])

diff = high - low

print(diff)

# 0 800

# 1 700

# 2 2000

# 3 700

# 4 800

# dtype: int64

```

- 같은 인덱스를 기준으로 값끼리 연산
- 반복문 없이 한 줄로 처리 가능

5 - 2. 인덱스가 다른 경우 → NaN 발생

사칙연산 — 인덱스 불일치

```

high = Series(data=[51500, 51200, 52500],

index=['5/1', '5/2', '5/3'])

low = Series(data=[50700, 50500, 50500],

index=['5/1', '5/2', '5/4']) # 5/4가 다름!

diff = high - low

print(diff)

# 5/1 800.0

# 5/2 700.0

# 5/3 NaN < - high에만 있음

```

5/4 NaN < - low에만 있음

상황	결과
양쪽 모두 인덱스 존재	정상 연산 수행
한쪽에만 인덱스 존재	NaN (Not a Number) — 계산 불가

[주의] NaN(Not a Number): 한쪽에만 존재하는 인덱스는 계산할 수 없어 NaN으로 표시됩니다. 데이터 분석 시 NaN 처리가 중요합니다.

6. 비교 연산 (Boolean Series)

[개념] 비교 연산의 결과는 True/False(Boolean) 타입이 저장된 새로운 Series가 리턴됩니다. 이 결과를 필터링에 활용할 수 있습니다.

비교 연산 예제

```
from pandas import Series

s = Series(data=[100, 200, 300, 400, 500])

cond = s > 300

print(cond)

# 0 False
# 1 False
# 2 False
# 3 True
# 4 True

# dtype: bool
```

- 비교 연산자 사용: >, <, >=, <=, ==, !=
- 결과는 Boolean(bool) 타입의 새로운 Series
- 이 결과를 필터링에 바로 활용 가능

7. 시리즈 필터링

7 - 1. Boolean 리스트로 필터링

Boolean 리스트 필터링

```
from pandas import Series

s = Series(data=[100, 200, 300, 400, 500])

cond = [False, False, False, True, True]

print(s[cond])

# 3 400

# 4 500

# dtype: int64
```

- True인 위치의 값만 추출됨

7 - 2. 비교 연산 결과로 필터링 (실전)

비교 결과 필터링

```
s = Series(data=[100, 200, 300, 400, 500])

cond = s > 300 # 비교 연산 -> Boolean Series

print(s[cond])

# 3 400

# 4 500

# 한 줄로 줄이기 (동일한 결과)

print(s[s > 300])
```

7 - 3. 두 Series를 활용한 교차 필터링

교차 필터링 예제

```
시가 = Series([100, 200, 300, 400, 600])
```



```
종가 = Series([100, 300, 400, 500, 600])
```

```
# 시가가 300 이상인 날의 종가 출력
```

```
조건 = (시가 >= 300)
```

```
print(종가[조건])
```

```
# 2 400
```

```
# 3 500
```

```
# 4 600
```

```
# dtype: int64
```

[핵심] 조건 시리즈(시가 기준)를 만들고, 그 조건을 다른 시리즈(종가)에 적용하여 원하는 데이터만 추출할 수 있습니다.

8. 다중 조건 필터링 (&, |, ~)

[중요] 판다스에서는 파이썬의 and/or/not 대신 &/|/~ 연산자를 사용합니다. 각 조건을 반드시 괄호()로 감싸야 합니다!

다중 조건 필터링 예제

```
from pandas import Series

종가 = Series([93000, 82400, 99100, 81000, 72300],
              index=['05/14','05/15','05/16','05/17','05/18'])

# 종가가 80000 이상 AND 90000 미만인 날짜 출력

cond = (종가 >= 80000) & (종가 < 90000) # & = AND

print(종가[cond].index)

# Index(['05/15', '05/17'], dtype='object')

# for문으로 날짜 하나씩 출력

for date in 종가[cond].index:

    print(date)

# 05/15

# 05/17
```

파이썬 논리연산	판다스 연산자	설명
and	&	두 조건 모두 True일 때
or		하나 이상 True일 때
not	~	조건을 반전 (True -> False)

[주의] [주의] 각 조건을 반드시 괄호로 감싸야 합니다! 예) (종가 >= 80000) & (종가 < 90000) - - 괄호 필수

9. 주요 개념 정리

개념	문법/설명
값 추가	<code>s.loc['목'] = 400 / s['금'] = 500</code>
값 삭제	<code>s.drop('월')</code> - - 원본 유지, 새 시리즈 리턴
원본 삭제	<code>s.drop('월', inplace=True)</code> - - 원본 직접 변경
다중 삭제	<code>s.drop(['월', '화'])</code> - - 리스트로 여러 개 동시 삭제
값 수정	<code>s.loc['수'] = 5000 / s.iloc[2] = 4000</code>
브로드캐스팅	<code>s + 10</code> - - 모든 원소에 동시 연산, 반복문 불필요
사칙연산	<code>high - low</code> - - 같은 인덱스끼리 연산
NaN	인덱스 불일치 시 계산 불가 - > NaN 발생
비교연산	<code>s > 300</code> - - Boolean Series 리턴
필터링	<code>s[cond]</code> - - True인 값만 추출
다중조건	(조건1) & (조건2) - - and는 &, or는 , not은 ~

[실습] 실습 팁: 주피터 노트북에서 각 코드를 직접 실행해 보세요. 추가/수정의 차이(인덱스 유무), drop의 inplace 옵션, 다중 조건의 괄호를 꼭 확인하세요!